Digital Twin-Driven Deep Reinforcement Learning for Energy-Aware CNC Milling Parameter Optimization

ME6324 – AI in Manufacturing

Course Project Report

Abhinand
Department of Mechanical Engineering
Indian Institute of Technology Madras
abhinand@iitm.ac.in

October 21, 2026

Git Revision: `d76603b` (main branch)

This report is generated from the project repository at `G:/My Drive/IITM/AI in Mfg`. All results are reproducible from the committed artifacts using the provided scripts.

Contents

1 Introduction **3**
- 1.1 Problem Statement . . . 3
- 1.2 Proposed Approach . . . 3
- 1.3 Contributions . . . 4

2 Related Work **4**

3 Digital Twin Physics **5**
- 3.1 Geometry and Kinematics . . . 5
- 3.2 Tool Wear (Extended Taylor Model) . . . 5
- 3.3 Surface Roughness (Empirical Regression) . . . 5
- 3.4 Energy Consumption . . . 5
- 3.5 Stochastic Tool Breakage . . . 6

4 Environment Design **6**
- 4.1 Registrations . . . 6
- 4.2 State Space . . . 6
- 4.3 Action Space . . . 7
- 4.4 Reward Function . . . 7
- 4.5 Episode Logic . . . 7
- 4.6 Domain Randomization . . . 8

5 Reinforcement Learning **8**
- 5.1 Algorithm . . . 8
- 5.2 Training Runs . . . 8
- 5.3 Monte Carlo Evaluation . . . 9

6 Classical Baselines **9**

7 Results **9**
- 7.1 Production Policy – Monte Carlo Sweep (100 Scenarios) . . . 10
- 7.2 Energy Weight Ablation – Pareto Frontier . . . 10
- 7.3 Runtime Weight Sweep of the Production Policy . . . 11
- 7.4 One Policy against the Per-Weight Policies . . . 11
- 7.5 Baseline Comparison . . . 12
- 7.6 Controller Initialization Experiment . . . 12
- 7.7 Transfer Experiment . . . 13

8 Interactive Digital Twin Workstation **14**

9 Software Engineering Practices **14**
- 9.1 Architecture . . . 14
- 9.2 Behavior Cloning as an Initialization . . . 15
- 9.3 Reproducibility . . . 15
- 9.4 Testing . . . 15
- 9.5 Code Quality . . . 15

10 Discussion **16**
- 10.1 What Worked . . . 16
- 10.2 Limitations . . . 16
- 10.3 Future Work . . . 17

11 Conclusion **17**

A Reproducibility Commands **18**

B Project Structure **19**

C Hyperparameter Sensitivity **20**

D Energy Model Derivation **21**

1 Introduction

1.1 Problem Statement

CNC face milling is a fundamental manufacturing operation where a rotating cutter removes material from a workpiece in multiple passes. The cutting parameters – spindle speed (n), feed per revolution (f_r), and axial depth of cut (a_p) – must be chosen to balance four competing objectives:

1. **Productivity:** Maximize material removal rate (MRR $= v_c \cdot a_p \cdot f_r$)
2. **Energy Efficiency:** Minimize total electrical energy consumption (E_{total}), accounting for both machine idle power and cutting power.
3. **Surface Quality Assurance:** Strictly adhere to finished workpiece roughness requirements ($R_a \leq 1.6$ μm).
4. **Tool Wear and Breakage Avoidance:** Limit cumulative flank wear ($V_B \leq 0.3$ mm) and mitigate catastrophic tool failure risks.

Traditional approaches use fixed parameters from handbooks or static optimization (e.g., NSGA-II) that pick one parameter set for the entire batch. These cannot adapt to the *state* of the process – tool wear accumulates, workpiece geometry changes, and material properties vary between batches.

1.2 Proposed Approach

This project develops a **Digital Twin** of the milling process – a high-fidelity physics-based simulator grounded in empirical manufacturing literature – and trains a **Deep Reinforcement Learning (DRL)** agent to dynamically optimize cutting parameters pass-by-pass. The agent observes the current state (flank wear, remaining stock, previous parameters) together with the context it is optimizing for (the energy weight and the material), and selects the next pass parameters to maximize a reward that balances throughput and energy while respecting constraints.

Key innovations:

- **Speed-dependent specific cutting energy** $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$ creates a genuine trade-off: faster cutting increases energy per unit volume but reduces idle energy.
- **Domain randomization** during training: material/tool coefficients, workpiece geometry, and initial wear are sampled per episode from literature-calibrated ranges.
- **One environment, one policy:** The energy weight w_2 and the material context live in the observation, so a single PPO policy covers the entire Pareto frontier and a whole distribution of materials.
- **Transfer experiment:** Measures how much a policy trained on one material knows about a harder, faster-wearing material.
- **Receding-horizon controller baseline:** A strong classical benchmark that re-solves at every pass.

1.3 Contributions

1. A Gymnasium-compliant milling environment with explicit physics, energy-aware reward, and domain randomization.
2. A trained PPO policy evaluated via 100-scenario Monte Carlo sweep with 95% confidence intervals.
3. Energy-weight ablation study (7 weights × 3 seeds) mapping the empirical Pareto frontier.
4. Comparison against 5 classical baselines, all on the same episodes:
   Constant-High, Constant-Safe, Rule-Based, NSGA-II Static, Receding-Horizon.
5. Transfer experiment on a harder alloy: a policy trained across a distribution of materials needs no adaptation there, while a policy trained on the nominal material violates the roughness limit until it is fine tuned.
6. Interactive Streamlit workstation with 2D stock depletion, micro-topography, G-code generation, and multi-controller comparison.
7. Full reproducibility: all results regenerated from committed artifacts by `experiments/summarize_results.py`.

2 Related Work

This project positions itself against the papers held in `docs/papers/`:

Table 1: Position against the two reference papers held in `docs/papers/`

Capability	Mo et al. (2025) *J. Manuf. Proc. 156*	Wang et al. (2025a) *J. Manuf. Sys. 78*	**This Work**
Dynamic parameter optimization	✓	✓	✓
Tool wear in the state	✓	✗	✓
Surface roughness constraint	✓	✗	✓
Energy as an objective	✗	✗	✓
Speed-dependent cutting energy	✗	✗	✓
Domain randomization	✗	✗	✓
Conditioned policy (single → frontier)	✗	✗	✓
Transfer experiment across materials	✗	✗	✓
Receding-horizon baseline	✗	✗	✓
Monte Carlo robustness (100 scenarios)	✗	✗	✓
Interactive digital twin workstation	✗	✗	✓

Both reference papers treat energy as related work rather than as an optimization target: in each of them energy appears in the discussion of prior art, not among the objectives. Mo et al. optimize the surface roughness and the production time of a multi-pass turning operation and carry the tool condition and the tool damage in the decision variables, so the wear and the roughness marks carry over. Wang et al. (2025a) build their state from sequences of machining parameters and cutting force features, and their objectives from the dimensional error of the machining features and the machining time; neither the tool wear nor the roughness enters their formulation, so only the dynamic parameter optimization carries over. The third paper in `docs/papers/` (Wang et al., 2025b) solves the discrete step sequencing problem, so continuous cutting parameters are outside its scope.

The speed-dependent cutting energy model (Section 3) is the key enabler here: without it the energy of a part is fixed by the volume removed, both objectives point the same way,

and no weight on the energy term can open a trade-off. Putting the energy weight and the material context in the observation reduces the ablation cost from 21 training runs to one. The receding-horizon controller provides a fair classical benchmark that uses the same physics and re-optimizes at every pass.

3 Digital Twin Physics

All equations are implemented in `MillingDT/config/physics.py` and used by the environment, baselines, simulator, and evaluation scripts – a single source of truth. The state the agent sees is the 11-dimensional observation of Section 4.2: eight physics features followed by the conditioning context, and the context is present in every task, so the equations below are the same equations for every task.

3.1 Geometry and Kinematics

Cutter diameter $D = 50$ mm, cutting length $L_{\text{cut}} = 100$ mm, total machining allowance $\kappa_{\text{total}} = 5.0$ mm.

$$v_c = \frac{\pi D n}{1000} \qquad [\text{m/min}] \tag{1}$$

$$\text{MRR} = v_c \cdot a_p \cdot f_r \qquad [\text{mm}^3/\text{min}] \tag{2}$$

$$t_{\text{cut}} = \frac{L_{\text{cut}}}{n f_r} \qquad [\text{min}] \tag{3}$$

$$N_{\text{passes}} = \left\lceil \frac{\kappa_{\text{total}}}{a_p} \right\rceil \qquad [\text{integer}] \tag{4}$$

3.2 Tool Wear (Extended Taylor Model)

Flank wear rate per pass:

$$\Delta V_B = C_{\text{wear}} \cdot v_c^{\alpha} \cdot f_r^{\beta} \cdot a_p^{\gamma_{\text{wear}}} \cdot t_{\text{cut}} \tag{5}$$

with calibrated coefficients $C_{\text{wear}} = 1.1 \times 10^{-4}$, $\alpha = 0.3$, $\beta = 0.2$, $\gamma_{\text{wear}} = 0.15$. Cumulative wear $V_{B,k} = \sum_{i=1}^{k} \Delta V_{B,i}$.

3.3 Surface Roughness (Empirical Regression)

$$R_a = k_1 f_r^2 + k_2 V_B + \frac{k_3}{v_c} \tag{6}$$

with $k_1 = 12.5$, $k_2 = 0.8$, $k_3 = 0.5$. The finished surface (last pass) must satisfy $R_a \leq R_{a,\max} = 1.6$ μm.

3.4 Energy Consumption

Specific cutting energy grows with cutting speed:

$$k_c(v_c) = k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^{m}, \quad P_{\text{cut}} = k_c(v_c) \cdot \text{MRR}, \quad P_{\text{total}} = P_{\text{idle}} + P_{\text{cut}} \tag{7}$$

with $P_{\text{idle}} = 500$ W, $k_{c0} = 2.5$ W $\cdot$ min/mm^3, $m = 0.6$, v_{ref} from the reference cutting condition. Energy per pass: $E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}}$. Total energy per part: $E_{\text{part}} = \sum_{k=1}^{N_{\text{passes}}} E_{\text{pass},k}$.

Proposition 1 (Interior Minimum)**.** *With $m > 0$, the energy per part $E_{part}(v_c)$ has an interior minimum in v_c. The cutting term rises with v_c while the idle term falls, creating a genuine trade-off that the energy weight w_2 can select.*

This is validated by test `test_part_energy_has_an_interior_minimum_in_speed`.

3.5 Stochastic Tool Breakage

Probability of catastrophic breakage per pass, applied only once the flank wear passes a threshold:

$$p_{\text{break}} = 0.3 \cdot \underbrace{\frac{V_B - 0.15}{V_{B,\max} - 0.15}}_{\text{wear factor}} \cdot \underbrace{\left(\frac{\text{MRR}}{\text{MRR}_{\max}}\right)^2}_{\text{load factor}} \qquad (V_B > 0.15 \text{ mm, else } 0) \tag{8}$$

where $\text{MRR}_{\max}$ is the removal rate of the machine envelope at its upper bounds. The draw comes from the environment's own generator, which `reset(seed=...)` seeds, so a scenario replayed with the same seed breaks in the same place. If triggered, the episode terminates with a -100 reward.

4 Environment Design

4.1 Registrations

One Gymnasium environment is registered in `MillingDT/env/__init__.py`, and it is exported as the constant `MillingDT.env.ENV_ID`:

1. `MillingDTEnv-v1`: the only registration. The observation is always $\mathbb{R}^{11}$: the eight physics features followed by the conditioning context (the energy weight w_2, the wear coefficient C_{wear} and the roughness coefficient k_1). The flags `randomize` and `randomize_weight` decide whether the scenario and the energy weight are drawn per `reset()`. A task with a pinned weight is the same environment with a constant in the context, so there is a single observation shape across the whole project.

4.2 State Space

Table 2: Observation (11 dimensions), every feature mapped onto $[0, 1]$

Index	Feature	Normalization
0	Flank wear V_B	tanh squash over $[0, V_{B,\max}]$
1	Remaining stock κ_{rem}	tanh squash over $[0, \kappa_{\text{total}}]$
2	Previous spindle speed n	$(n - n_{\min})/(n_{\max} - n_{\min})$
3	Previous feed f_r	$(f_r - f_{\min})/(f_{\max} - f_{\min})$
4	Previous depth a_p	$(a_p - a_{p,\min})/(a_{p,\max} - a_{p,\min})$
5	Previous MRR	$\text{MRR}/\text{MRR}_{\max}$, clipped
6	Previous roughness R_a	tanh squash over $[0, R_{a,\max}]$
7	Tool broken	1 if broken, else 0
8	Energy weight w_2	$w_2 \in [0, 1]$
9	Wear coefficient C_{wear}	over the scenario band $[0.6, 1.4]$ of nominal
10	Roughness coefficient k_1	over the scenario band $[0.8, 1.2]$ of nominal

Every feature is normalized against the *scenario's own* limits, so the observation is scale free across geometries and materials. The tanh squashing keeps the gradient magnitude when a constraint is violated, instead of clipping the value to 1.

The context occupies the last three slots in every task. A policy trained with a pinned weight sees a constant there, a policy trained with the weight drawn sees the weight it is being asked for, and a policy trained across the material distribution sees the material it is cutting. The shape never changes, so one trained policy can be asked for any weight and any material at runtime.

4.3 Action Space

Continuous 3D box $\mathcal{A} = [-1, 1]^3$, mapped to physical parameters:

$$n = n_{\min} + \frac{a_1 + 1}{2}(n_{\max} - n_{\min}) \tag{9}$$

$$f_r = f_{\min} + \frac{a_2 + 1}{2}(f_{\max} - f_{\min}) \tag{10}$$

$$a_p = a_{p,\min} + \frac{a_3 + 1}{2}(a_{p,\max} - a_{p,\min}) \tag{11}$$

The policy output is unbounded; the environment clips to $[-1, 1]$ before mapping, guaranteeing the machine envelope is never violated.

4.4 Reward Function

The per-step reward is:

$$\begin{aligned} r = {} & \underbrace{5\left(w_1 \frac{\mathrm{MRR}}{\mathrm{MRR}_{\max}} - w_2 \frac{E_{\mathrm{pass}}}{E_{\mathrm{ref}}}\right)}_{\text{Objective}} + \underbrace{2\,\Delta\left(\frac{\kappa_{\mathrm{removed}}}{\kappa_{\mathrm{total}}}\right)}_{\text{Progress}} \\ & - \underbrace{50\mathbb{1}[V_B > V_{B,\mathrm{safe}}]}_{\text{Wear penalty}} - \underbrace{100\mathbb{1}[\text{broken}]}_{\text{Breakage}} \\ & - \underbrace{50\mathbb{1}[\text{finishing pass} \wedge R_a > R_{a,\mathrm{safe}}]}_{\text{Roughness penalty}} - 0.05 \end{aligned} \tag{12}$$

plus, on a part finished inside spec, a +200 completion bonus and a Pareto margin bonus of at most +5:

$$\mathrm{HV} = \max\left(0, 1 - \tfrac{T_{\mathrm{total}}}{T_{\mathrm{ref}}}\right) \cdot \max\left(0, 1 - \tfrac{R_a}{R_{a,\mathrm{safe}}}\right) \cdot 5 \tag{13}$$

The objective term carries a scale factor of 5 so that it competes with the shaping terms. Without it the progress bonus and the constraint penalties decide the policy on their own, and the energy weight w_2 has no visible effect at all: an earlier revision of the reward produced 21 identical ablation policies. The constraint penalties and the completion bonus are an order of magnitude larger on purpose, so that breaking a constraint is never worth a throughput or energy gain. $R_{a,\mathrm{safe}} = 0.85\,R_{a,\max}$, $w_1 + w_2 = 1$, and E_{ref} is the energy of a pass under the reference condition, recomputed for every scenario from that scenario's diameter, cutting length and energy coefficients.

The reward is validated by `test_reward_matches_an_independent_calculation`, which recomputes it from `MillingDT/config/physics.py`, independently of the code that assembles it, and asserts agreement to 10^{-9}.

4.5 Episode Logic

- `terminated=True`: Part finished ($\kappa_{\mathrm{rem}} \leq 0$), tool broken, or wear exceeded.
- `truncated=True`: Maximum steps (200) reached.
- The roughness penalty is charged once, at the finishing pass, and only when that pass ends outside $R_{a,\mathrm{safe}}$. Intermediate passes carry no roughness penalty; the wear penalty is charged on every pass whose cumulative flank wear exceeds $V_{B,\mathrm{safe}}$.

4.6 Domain Randomization

When `randomize=True`, each `reset()` samples a new scenario from `MillingDT/config/scenarios.py`:

- Workpiece: $\kappa_{\text{total}} \in [2, 15]$ mm, $D \in [40, 150]$ mm, $L_{\text{cut}} \in [50, 200]$ mm
- Material/Tool: C_{wear} at $[0.6, 1.4]\times$ nominal, k_1, k_2, k_3 each at $[0.8, 1.2]\times$ nominal
- Initial wear: $V_{B,0} \sim \mathcal{U}[0, 0.8]\, V_{B,\max}$

The same tables are used for Monte Carlo evaluation, so the policy is measured on the distribution it was trained on. Measured on the trained policy, the eight physics features the policy receives on the two sets agree to 0.041 in the mean of their normalized values, against a standard deviation of 1.

5 Reinforcement Learning

5.1 Algorithm

Proximal Policy Optimization (PPO) with MLP policy, implemented via Stable-Baselines3.

Table 3: PPO hyperparameters

Parameter	**Value**
Policy	MlpPolicy (2 hidden layers, 256 units each)
Learning rate	3×10^{-4} (linear schedule to 0)
n_steps	2048
Batch size	64
n_epochs	10
γ	0.99
GAE λ	0.95
Clip range	0.2
Entropy coefficient	0.02
Value function coefficient	0.5
Max grad norm	0.5
VecNormalize	`norm_obs=True, norm_reward=True`
Environments	4 parallel (`SubprocVecEnv`)
Device	CPU (2.5× faster than GPU for this MLP)

Training uses a custom `ProgressQueueCallback` that sends a heartbeat every 1000 steps to the supervisor (`run_experiments.py`); a dropped heartbeat triggers worker termination.

5.2 Training Runs

1. **Production policy:** 1,000,000 steps, seed 42, `--randomize --randomize-weight`. Both the machining scenario and the energy weight are drawn per episode, so one network covers the whole weight range and the whole material distribution. Model at `results/run_PPO_seed42/`. This is the policy the dashboard, the Monte Carlo sweep, the runtime weight sweep and the baseline comparison all use.

2. **Ablation grid:** 7 weights $w_2 \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}\times$ 3 seeds $\{42, 43, 44\}$, 150,000 steps each, trained in parallel via `experiments/run_experiments.py`. Every ablation policy is trained with its energy weight **pinned** and is evaluated at the weight it

was trained for, so it is always asked the question it was trained for. The model directories follow the pattern `results/run_ablation_w2_0_4_seed42`, one per weight and seed.

Both training paths use the same script, `experiments/train_rl.py`: `--randomize` draws a new machining scenario per episode, `--randomize-weight` draws a new energy weight per episode, and `--w2` pins the weight of the task.

5.3 Monte Carlo Evaluation

`python -m experiments.evaluate --scenarios 100` runs the production policy over 100 randomized scenarios (fixed seed 42). Each scenario is a full machining episode. Reports mean ± std and 95% CI (t-distribution) for:

- MRR, Energy/part, Cycle time
- Max R_a, Max V_B
- Constraint violation rates (worst pass vs finished surface)
- Tool breakage rate, Part completion rate

6 Classical Baselines

Five non-RL baselines implemented in `experiments/run_baselines.py`:

1. **Constant-High:** $n = 8000, f_r = 0.35, a_p = 2.5$ (upper bounds).
2. **Constant-Safe:** $n = 2000, f_r = 0.05, a_p = 0.1$ (lower bounds).
3. **Rule-Based Degradation:** Feed at maximum, spindle speed reduced by 20% every second pass, as a shop-floor heuristic for tool degradation.
4. **NSGA-II Static:** Multi-objective optimization (pymoo) picking *one* parameter set for the whole part. Decision variables $[n, f_r, a_p]$. Objectives: maximize MRR, minimize Energy. Constraints: $R_{a,\text{final}} \leq R_{a,\max}$, $V_{B,\text{final}} \leq V_{B,\max}$.
5. **Receding-Horizon Controller:** Vectorized grid search over (n, f_r, a_p) at *every pass*, simulating the remaining passes with the current V_B and κ_{rem}. Keeps feasible candidates, picks the best by the same scalarization $w_1\text{MRR} - w_2 E$. This is the strongest classical baseline.

All baselines use the *same physics* (`MillingDT/config/physics.py`) and are evaluated on the same 50 episodes of the nominal machining condition, at the same energy weight $w_2 = 0.4$ at which the learned policy is queried.

7 Results

All tables below are regenerated from committed CSVs by
`python -m experiments.summarize_results`.

7.1 Production Policy – Monte Carlo Sweep (100 Scenarios)

Table 4: Production policy Monte Carlo results (100 scenarios)

Metric	**Mean**	**Std**	**95% CI**
Material removal rate (mm^3/min)	1205.15	585.65	[1088.94, 1321.35]
Energy per part (kJ)	68.35	48.26	[58.77, 77.92]
Cycle time per part (s)	20.25	17.06	[16.86, 23.63]
Max surface roughness (μm)	1.379	0.350	[1.309, 1.448]
Max flank wear (mm)	0.1244	0.0679	[0.1109, 0.1378]
Constraint violations			
Roughness violations, worst pass	38.0%		[28.5%, 47.5%]
Roughness violations, finished surface	0.0%		[0.0%, 0.0%]
Roughness > safety limit, worst pass	56.0%		[46.3%, 65.7%]
Roughness > safety limit, finished surface	0.0%		[0.0%, 0.0%]
Flank wear violations ($V_B > 0.3$ mm)	0.0%		[0.0%, 0.0%]
Tool breakage	1.0%		[0.0%, 3.0%]
Parts completed within spec	99.0%		
Scenarios	100		
Mean operating point	7199 RPM, 0.247 mm/r, 2.11 mm, 4.0 passes		

Key observations:

- **Finished surface always within spec:** 0% roughness violations on the final pass.
- **Wear never exceeded:** 0% flank wear violations.
- **Intermediate roughness:** 56% of passes exceed the safety margin ($0.85 \times 1.6 = 1.36$ μm), but the policy recovers by the finish pass.
- **High variance:** Std of MRR is 49% of mean – domain randomization produces diverse operating points.

7.2 Energy Weight Ablation – Pareto Frontier

Table 5: Ablation study: mean ± std over 3 seeds, 50 scenarios each, every policy trained and evaluated with its weight pinned

w_2	MRR (mm^3/min)	Energy (kJ)	Cycle (s)	n (RPM)	f_r (mm/r)	Frontier
0.0	946.3 ± 63.4	82.9 ± 1.4	44.8 ± 2.4	7094	0.166	dominated
0.2	1007.8 ± 22.7	77.1 ± 1.1	31.3 ± 1.3	7471	0.184	dominated
0.4	1038.0 ± 67.6	72.7 ± 0.5	24.8 ± 1.8	7256	0.203	✓
0.5	1035.8 ± 43.8	70.3 ± 1.2	22.0 ± 0.5	7080	0.214	✓
0.6	1023.9 ± 27.4	70.8 ± 0.6	22.1 ± 0.5	7148	0.209	dominated
0.8	971.1 ± 43.2	66.8 ± 1.2	21.9 ± 0.7	6538	0.228	✓
1.0	951.3 ± 54.7	65.8 ± 0.6	20.9 ± 0.6	6490	0.229	✓

Proposition 2 (Frontier Shape)**.** *Across the Pareto frontier ($w_2 \in [0.4, 1.0]$):* MRR *falls 8.4% (1038 → 951 mm^3/min) while energy falls 9.5% (72.7 → 65.8 kJ). The trade-off is symmetric: the energy of the part falls slightly more than the removal rate, and the spindle speed falls with the energy weight, which is the mechanism the model predicts.*

The dominated points ($w_2 = 0.0, 0.2, 0.6$) all spend more energy than a point on the frontier. The two low weights push the speed too high, which raises $k_c(v_c)$ without cutting the idle energy enough to compensate. The $w_2 = 0.6$ point is dominated by $w_2 = 0.5$, which removes more material for less energy.

7.3 Runtime Weight Sweep of the Production Policy

The weight is a query, not a retrain: the same policy is asked for each point of the trade-off at runtime (`python -m experiments.evaluate --sweep-weight`) over the same 100 scenarios.

Table 6: Production policy asked for every weight at runtime (100 scenarios)

w_2	MRR (mm^3/min)	Energy (kJ)	Cycle (s)	n (RPM)	f_r (mm/r)	R_a violation rate
0.0	1213.6	69.6	19.0	7513	0.238	34.0%
0.2	1215.3	69.0	19.5	7379	0.243	36.0%
0.4	1205.1	68.3	20.2	7199	0.247	38.0%
0.5	1191.0	68.0	20.9	7081	0.249	38.0%
0.6	1169.3	67.6	21.5	6953	0.250	39.0%
0.8	1130.8	67.1	23.0	6710	0.251	39.0%
1.0	1096.0	67.1	24.8	6475	0.249	39.0%

As the weight goes from 0.0 to 1.0 the removal rate falls from 1213.6 to 1096.0 mm^3/min, the energy of the part falls from 69.6 to 67.1 kJ, the cycle time rises from 19.0 to 24.8 s and the spindle speed falls from 7513 to 6475 RPM. The operating point moves along the trade-off without a retrain.

7.4 One Policy against the Per-Weight Policies

The production policy is compared against the 21 ablation policies, one per weight, each trained and evaluated with its weight pinned (`python -m experiments.evaluate --sweep-weight --compare-only`).

Table 7: One policy against 21 policies, one per weight. Removal rate in mm^3/min, energy per part in kJ.

w_2	MRR, one policy	MRR, per weight	MRR gap	Energy, one policy	Energy, per weight	Energy gap
0.0	1213.6	946.3	+28.2%	69.6	82.9	−16.0%
0.2	1215.3	1007.8	+20.6%	69.0	77.1	−10.5%
0.4	1205.1	1038.0	+16.1%	68.3	72.7	−6.0%
0.5	1191.0	1035.8	+15.0%	68.0	70.3	−3.3%
0.6	1169.3	1023.9	+14.2%	67.6	70.8	−4.5%
0.8	1130.8	971.1	+16.4%	67.1	66.8	+0.5%
1.0	1096.0	951.3	+15.2%	67.1	65.8	+2.1%

A single policy of 1,000,000 steps covers the whole weight range and beats the 21 weight-specific policies of 150,000 steps each, which is 3,150,000 steps in total. At every weight it removes 14% to 28% more material. From $w_2 = 0.0$ to $w_2 = 0.6$ it also spends up to 16% less energy, and at $w_2 = 0.8$ and $w_2 = 1.0$ it removes 16.4% and 15.2% more material for 0.5% and 2.1% more energy. The energy weight stays a runtime query rather than a retrain.

7.5 Baseline Comparison

All six policies are evaluated on the same 50 episodes of the nominal machining condition at the same energy weight $w_2 = 0.4$, so the numbers are comparable. The reward column is the decisive one, because it carries the constraint penalties.

Table 8: Baseline comparison, 50 episodes of the nominal condition at $w_2 = 0.4$

Policy	MRR **(mm^3/min)**	**Energy (kJ)**	**Cycle (s)**	**Mean reward**
PPO (ours)	894.3	18.40	5.56	**+204.09**
Receding-Horizon	**981.7**	**18.02**	**4.80**	**+203.45**
NSGA-II Static	732.9	18.8	6.4	−46.39
Rule-Based	719.8	18.7	6.6	−46.48
Constant-High	1099.6	17.8	4.3	−46.10 (violates)
Constant-Safe	1.6	1506.8	3000.0	−139.25

- **The four static policies finish outside the roughness safety limit** ($R_{a,\text{safe}} = 0.85 \times 1.6 = 1.36$ μm), which the reward penalizes with −50. The receding-horizon controller is the only classical policy that stays inside it, and it reaches +203.45 against −46 for the rest.
- **The learned policy and the controller are level on the reward** (+204.09 against +203.45), while the controller is ahead on the removal rate (981.7 against 894.3 mm^3/min) and on the cycle time (4.80 s against 5.56 s). The energy of the part is close (18.02 kJ against 18.40 kJ).
- **This digital twin does not yet show a reinforcement learning advantage.** The next experiment initializes the policy from the controller and fine-tunes from there (Section 7.6).

7.6 Controller Initialization Experiment

The baseline comparison left one question open: the controller re-solves the next pass with the exact state and the exact physics, and the learned policy is level with it on the reward but behind it on the physical metrics. The experiment asks whether a policy that starts from the controller ends up better than the expert it started from.

The method is a behavior cloning step used as an initialization. The controller's behavior is recorded on 2000 randomized episodes, a supervised actor with the same shape as the PPO actor (two hidden layers of 256 with Tanh) is fitted on those trajectories by regression, and the fitted weights are copied into a fresh PPO policy. The reinforcement learning therefore starts from a feasible behavior instead of from noise. The policy is then trained with the same budget as the production policy (1,000,000 steps) and evaluated on the same 100 randomized scenarios at the same energy weight $w_2 = 0.4$. The clone lives in `MillingDT/agent/controller_init.py`, the experiment in `experiments/controller_experiment.py`, and the trained policy in `results/run_PPO_controller_init_seed42/`.

Table 9: Controller initialization experiment: three arms on the same 100 scenarios at $w_2 = 0.4$. Std is the standard deviation of the episode reward over the scenarios.

Arm	**Mean reward**	**Std**	MRR (mm^3/min)	**Energy** (kJ/part)	**Cycle** (s)	R_a **viol.** finished	**Tool** breakage	**Parts** completed
Receding-horizon controller	187.96	65.92	**1560.6**	68.01	**13.54**	0.0%	5%	95%
PPO from scratch	198.57	30.03	1205.1	68.35	20.25	0.0%	1%	99%
PPO initialized from the controller	**199.35**	**3.35**	1126.1	77.17	32.35	0.0%	**0%**	**100%**

The initialization does not produce a policy that beats the controller. The controller still removes 38% more material per minute (1560.6 against 1126.1 mm^3/min) and finishes in 13.5 s against 32.3 s. The reward cannot separate the two learned policies either: 199.35 against 198.57 is a gap of 0.8, and the standard error of the mean over the 100 scenarios is $30.0/\sqrt{100} = 3.0$ for the from-scratch arm. A gap of 0.8 is inside the noise, so it is not a result.

What it does produce is consistency. The spread of the episode reward across the 100 scenarios falls from 30.0 to 3.4, the tool breakage falls to zero, and every part is completed. The controller itself is the least reliable of the three arms, with a reward std of 65.9, 5% tool breakage and 95% of the parts completed.

The consistency is paid for with throughput and energy. Against the from-scratch policy the initialized policy removes 6.6% less material per minute and spends 12.9% more energy per part. Starting from the feasible behavior of the controller gives a policy that scores almost the same on every scenario, and that operating point is slower and more expensive than the one a policy started from noise finds.

7.7 Transfer Experiment

Target material: harder alloy ($k_1 \times 1.25$, $k_2 \times 1.30$) with faster-wearing tool ($C_{\text{wear}} \times 2.0$). The source policy was trained on the nominal material only and has never seen this material.

Table 10: Transfer experiment on the target material, 30 scenarios

Arm	**Mean reward**	R_a **violation worst pass**	R_a **violation finished surface**	**Parts completed**
Source policy, zero-shot	182.5	100.0%	3.3%	93.3%
Source policy, fine-tuned (50k steps)	196.6	0.0%	0.0%	100.0%
Trained on the target from scratch (50k steps)	196.6	0.0%	0.0%	100.0%
Production policy, zero-shot	**200.3**	60.0%	0.0%	100.0%

The production policy needs no adaptation. It was trained across a distribution of materials with the material context in its observation, so the harder alloy is inside its training distribution. Zero shot it scores the highest reward of the four arms (+200.3), the finished surface, which is the part quality the reward constrains, is inside the limit in every one of the 30 scenarios, and every part is completed. The 60% worst-pass violations are on the roughing passes, which the reward does not constrain.

The nominal-trained source policy cannot do this. Zero shot it scores 182.5, violates the roughness limit in every scenario, finishes outside the limit in 3.3% of them and completes only 93.3% of the parts. Fine tuning it for 50,000 steps on the target removes every violation

and lifts the reward to 196.6, and training on the target from scratch for the same budget lands at exactly the same place, 196.6. A material change can therefore be repaired by adaptation, but the policy that already carries the material in its state does not need the repair: it is the best of the four arms without a single adaptation step.

8 Interactive Digital Twin Workstation

The Streamlit application (`app.py`) provides an industrial-grade interface:

1. **2D Cross-Sectional Stock Depletion:** Real-time visualization of cylindrical billet, pass-by-pass cutout, remaining allowance κ, depth per pass a_p, and tool engagement.
2. **Micro-Topography Generator:** Feed-mark cusp profile with an ISO 1302 centerline average R_a overlay and a 1.6 μm ceiling:

$$y_{\text{cusp}}(x) \approx \frac{x^2}{8r_\epsilon} + \text{stochastic perturbation}$$

3. **ISO 6983 G-Code Synthesizer:** Emits Fanuc/Siemens compliant G-code for the exact PPO parameter sequence.
4. **Multi-Controller Comparison:** PPO against NSGA-II, Rule-Based and Constant-High, with a telemetry overlay.
5. **Cached Policy Loading:** `@st.cache_resource` ensures model loads once per session.

The workstation loads the production policy (`results/run_PPO_seed42`) and exposes the energy weight as a runtime slider, so the whole trade-off is reachable from the browser without a retrain.

Launch: `streamlit run app.py`

9 Software Engineering Practices

9.1 Architecture

- `MillingDT/config/`: Calibrated constants and paths (YAML), the shared physics equations, the scenario distributions (single source of truth)
- `MillingDT/env/`: Gymnasium environment, one registration (`MillingDTEnv-v1`)
- `MillingDT/agent/`: The controller initialization, the behavior clone of the receding-horizon controller
- `MillingDT/baselines/`: The receding-horizon controller
- `MillingDT/sim/`: Simulation engine for Streamlit (stock depletion, micro-topography, G-code)
- `MillingDT/utils/`: Seeding, reproducibility
- `experiments/`: The entry points: training, evaluation, ablation, baselines, transfer, controller experiment, pipeline orchestration
- `viz/`: Plotting utilities, training curves, MC distributions, baseline comparison
- `results/`: One directory per run, holding the policy, the log, the metrics and the figures of that run

9.2 Behavior Cloning as an Initialization

The receding-horizon controller is a competent expert, so it can supply the starting point of the learner instead of leaving the learner to start from noise. The module `MillingDT/agent/controller_init.py` records what the controller does, fits a supervised actor of the same shape as the PPO actor on those trajectories by regression, and copies the fitted weights into a fresh PPO policy. The initialization stays a separate concern: the clone knows nothing about the training loop, the copy is a weight assignment between two tensors of the same shape, and the rest of the run is an ordinary PPO training, selected with the `--init-from-controller` flag. The same separation is what makes the arm comparable: the three arms of Table 9 share the environment, the scenarios, the budget and the evaluation code, so the only difference between them is where the weights start.

9.3 Reproducibility

- Global seeding via `MillingDT/utils/seeding.py` (Python, NumPy, PyTorch, CUDA deterministic)
- `MillingDT/config/scenarios.py` shared by training and Monte Carlo – no drift
- `experiments/summarize_results.py` rebuilds `RESULTS.md` from committed CSVs
- All random draws (scenarios, weights, seeds) are deterministic given the seed

9.4 Testing

34 tests in `tests/` covering:

- Environment: action clipping, truncation, soft norm, the 11-dimensional observation, physics constants, overrides, info dict, energy norm, Gym checker, randomization, pinned values, scenario scales, the context (weight and material), cutting energy speed dependence, part energy interior minimum, shared physics, shared scenario tables
- Reward: golden-value test (independent recomputation), finishing-pass penalty
- Integration: training→evaluation roundtrip, one registration, the context in the observation, one policy answering a pinned weight, baseline artifact
- Progress callback: heartbeat delivery, failure reporting, interval respect

9.5 Code Quality

- Centralized constants (`MillingDT/config/physics_constants.yaml`), one calibrated value per line with its unit in the file
- Centralized paths (`MillingDT/config/paths.yaml`) – no hardcoded strings
- Environment ID exported from `MillingDT/env/__init__.py`
- No unused imports (AST-verified)
- Conventional commits, clean git history

10 Discussion

10.1 What Worked

1. The speed-dependent cutting energy model ($k_c \propto v_c^m$) is essential. In an earlier revision of the reward, without it the energy/throughput trade-off vanished and the 21 ablation policies came out identical.

2. Domain randomization + shared scenario tables: the policy generalizes to unseen scenarios (99% part completion, 0% finished-surface violations).

3. Putting the energy weight in the observation reduces the ablation cost from 21 training runs to one, and the one policy beats the 21 per-weight policies at every weight.

4. Training across a distribution of materials removes the need for an adaptation step: on a harder alloy the production policy is the best of the four transfer arms zero shot, while a policy trained on the nominal material violates the roughness limit until it is fine tuned.

5. The receding-horizon controller is a strong, fair baseline using the same physics. It also exposed that the earlier baseline comparison was hollow.

6. Initializing the policy from the controller does not make it beat the controller, but it does make it uniform: the reward std over the 100 scenarios falls from 30.0 to 3.4, the tool breakage falls to zero and every part is completed (Table 9).

7. The test suite catches physics regressions (energy interior minimum, cutting energy speed dependence) and reward regressions (golden-value test).

10.2 Limitations

1. The learned policy does not outperform the receding-horizon controller on the reward (Table 8), and the controller is ahead on the physical metrics. The controller re-solves with perfect physics; the policy has to learn the dynamics from experience. Initializing the policy from the controller does not close that gap: the initialized policy is more consistent (reward std 3.4 against 30.0) but slower and more hungry for energy (6.6% less removal rate and 12.9% more energy per part against the from-scratch policy), and the difference in mean reward, 199.35 against 198.57, is inside the standard error of the mean of 3.0.

2. The energy weight ablation uses 150,000 steps per weight against 1,000,000 for the production policy, so the ablation frontier is a lower bound on what a pinned policy can reach and may shift with more training.

3. The reward charges the energy of a pass, while the reported metric is the energy of a part. A part cut in many shallow passes draws less power per pass but spends more time on the idle machine, so the two can move in opposite directions. Both are reported.

4. Stochastic tool breakage (1% rate) introduces variance; the reward penalty could be shaped more carefully.

5. The micro-topography model is a simplified feed-mark cusp; real chatter and vibration are not modeled.

6. The energy model is the modelling choice of this work, not a result taken from the reference papers, and it is calibrated against a single reference condition.

7. The frontier is symmetric but shallow: 8.4% less removal rate buys 9.5% less energy. A decision-maker who needs a large energy reduction has to pay for it in throughput elsewhere, and this digital twin does not yet show a way to escape that trade-off.

10.3 Future Work

1. **Against a restricted controller:** The controller re-solves every pass with the exact state and the exact physics, so it is an oracle that a learned policy cannot match from its observation alone. Restricting the controller's information, or pricing the cycle time explicitly in the reward, would tell whether the remaining gap is an information gap or an optimization gap.

2. **Sim-to-real:** Calibrate physics coefficients on real machine data; add sensor noise and delay to the environment.

3. **Multi-objective RL:** Replace scalarization with Pareto-conditioned networks or hypervolume maximization.

4. **Runtime energy budget:** Drive the production policy with an optimizer that sets w_2 from an energy budget. The weight is a query, not a retrain.

5. **Full 3D Digital Twin:** Extend the 2D cross-section to 3D toolpath visualization with collision checking.

11 Conclusion

This project demonstrates a complete pipeline for DRL-based CNC milling optimization:

1. A physics-grounded Digital Twin with speed-dependent cutting energy, tool wear, surface roughness, and stochastic breakage.

2. A Gymnasium environment with domain randomization, an always 11-dimensional conditioned observation, and an energy-aware reward whose objective terms are scaled to compete with the shaping terms.

3. PPO policies trained with proper experimental rigor: Monte Carlo evaluation with confidence intervals, an ablation study mapping the empirical Pareto frontier, and one production policy covering the frontier in a single run.

4. Strong classical baselines including a receding-horizon controller that re-optimizes at every pass.

5. A controller initialization experiment, which records the receding-horizon controller on 2000 episodes, clones its actor into a fresh PPO policy and trains it with the same 1,000,000-step budget. The initialized policy is much more uniform (reward std 3.4 against 30.0 over the 100 scenarios), breaks no tools and completes every part, but it does not beat the controller: the controller removes 38% more material per minute and finishes in 13.5 s against 32.3 s.

6. A transfer experiment on a harder alloy, which finds that a policy trained across a distribution of materials needs no adaptation there, while a policy trained on the nominal material violates the roughness limit until it is fine tuned.

7. An interactive workstation for visualization, G-code generation, and multi-controller comparison.

8. Full reproducibility: every result table is generated from committed artifacts by a single script.

The key scientific finding is that **the speed-dependent specific cutting energy creates a genuine energy/throughput trade-off**, and a DRL agent can learn to navigate it pass-by-pass while respecting surface quality and tool life constraints. Across the measured frontier the removal rate falls 8.4% while the energy of the part falls 9.5%, a symmetric trade-off, and the spindle speed falls with the energy weight, which is the mechanism the model predicts. One policy with the weight in its observation makes the whole trade-off accessible at runtime, and it beats the 21 per-weight policies at every point of it.

The result on the classical side is the opposite of an advantage: the receding-horizon controller still removes more material per minute and finishes sooner than the learned policy, and initializing the policy from it does not change that. What the initialization does buy is consistency, and it buys it with throughput and energy. The claim this work supports is therefore that the reinforcement learning agent navigates the energy/throughput trade-off within a fixed constraint set, not that it beats a classical controller that re-optimizes with perfect physics.

All code, data, and results are available in the repository at `G:/My Drive/IITM/AI in Mfg`.

References

References

[1] X. Mo, M. Xia, S. Mei, X. Hu, "Deep reinforcement learning-based optimization for machining process: Cutting parameters dynamic optimization considering tool condition degradation," *Journal of Manufacturing Processes* 156 (2025) 1006–1031.

[2] P. Wang, Y. Cui, H. Tao, X. Xu, S. Yang, "Machining parameter optimization for a batch milling system using multi-task deep reinforcement learning," *Journal of Manufacturing Systems* 78 (2025) 124–152, doi: 10.1016/j.jmsy.2024.11.013.

[3] P. Wang, Y. You, W. Liu, D. Ding, F. Xu, "Knowledge-guided deep reinforcement learning framework for feature machining step sequencing," *International Journal of Advanced Manufacturing Technology* 140 (2025) 5617–5636, doi: 10.1007/s00170-025-16606-4.

A Reproducibility Commands

All commands run from the project root `G:/My Drive/IITM/AI in Mfg`:

Listing 1: Key reproducibility commands

```
# Run all tests
pytest tests/ -v

# Train the production policy (1M steps, scenario and weight drawn per
    episode)
python -m experiments.train_rl --steps 1000000 --seed 42 --randomize --
    randomize-weight

# Train the production policy with the energy weight pinned
python -m experiments.train_rl --steps 150000 --seed 42 --randomize --
    w2 0.6

# Monte Carlo sweep of the production policy (100 scenarios)
python -m experiments.evaluate --scenarios 100

# Runtime weight sweep of the same policy (100 scenarios)
python -m experiments.evaluate --sweep-weight --scenarios 100
```

```
# Compare the saved weight sweep against the per-weight policies
python -m experiments.evaluate --compare-only

# Ablation study (7 weights x 3 seeds, 150k steps each, weight pinned)
python -m experiments.train_rl_ablation --steps 150000 --scenarios 50
    --slots 7

# Classical baselines
python -m experiments.run_baselines

# Transfer experiment (300k source steps, 50k adaptation steps)
python -m experiments.transfer_experiment 300000 50000

# Controller initialization experiment (1M steps, 2000 controller
    episodes)
python -m experiments.controller_experiment 1000000 2000

# Regenerate all plots
python -m viz.plot_results
python -m viz.plot_eval
python -m viz.plot_baselines_comparison

# Regenerate RESULTS.md from artifacts
python -m experiments.summarize_results

# Launch interactive workstation
streamlit run app.py

# Full pipeline (sequential)
python -m experiments.run_pipeline --steps 1000000 --ablation_steps
    150000
```

B Project Structure

The project holds one Gymnasium registration, `MillingDTEnv-v1`, one training script and one evaluation script. The registration is exported as the constant `MillingDT.env.ENV_ID`. There is no separate conditioned environment, no separate conditioned training script and no `results/conditioned/` directory. The reusable code lives in the `MillingDT/` package, the entry points in `experiments/`, and every training configuration keeps its own directory under `results/`.

Listing 2: Project directory tree

```
G:\My Drive\IITM\AI in Mfg
|-- MillingDT/                        # The library package
|   |-- __init__.py                    # Exports ENV_ID, ALL_CONSTANTS
|   |-- config/
|   |   |-- __init__.py                # Loads the two YAML files
|   |   |-- physics_constants.yaml     # The calibrated constants
|   |   |-- paths.yaml                 # Where the runs are kept
|   |   |-- physics.py                 # Shared physics equations
|   |   `-- scenarios.py               # Randomized scenario sampling
|   |-- env/
|   |   |-- milling_env.py             # Gymnasium environment (11-D
    observation)
```

```
|   |   `-- __init__.py              # Exports ENV_ID (one registration
    )
|   |-- agent/
|   |   |-- controller_init.py       # Behavior clone of the controller
|   |   `-- __init__.py
|   |-- baselines/
|   |   |-- receding_horizon.py      # Receding-horizon controller
|   |   `-- __init__.py
|   |-- sim/
|   |   |-- machining_sim.py         # Stock depletion, topography, G-
    code
|   |   `-- __init__.py
|   `-- utils/
|       |-- seeding.py               # Global deterministic seeding
|       `-- __init__.py
|-- experiments/                     # The entry points
|   |-- train_rl.py                  # The one training script
|   |-- train_rl_ablation.py         # Energy weight ablation, pinned
|   |-- evaluate.py                  # Monte Carlo sweep, weight sweep
|   |-- run_baselines.py             # 5 classical baselines
|   |-- transfer_experiment.py       # Cross-material transfer
|   |-- controller_experiment.py     # Controller initialization, 3
    arms
|   |-- run_experiments.py           # Parallel training supervisor
|   |-- run_pipeline.py              # Full pipeline orchestrator
|   |-- summarize_results.py         # Regenerates RESULTS.md
|   `-- __init__.py
|-- viz/
|   |-- plotting_utils.py            # Shared plotting functions
|   |-- plot_eval.py                 # Monte Carlo distributions
|   |-- plot_results.py              # Training curves
|   |-- plot_baselines_comparison.py # Baseline comparison charts
|   `-- __init__.py
|-- tests/
|   |-- test_env.py                  # Environment and reward tests
|   |-- test_training.py             # Training tests
|   |-- test_progress_callback.py    # Callback tests
|   `-- test_integration.py          # Integration tests
|-- results/                         # One directory per configuration
|   |-- run_<config>/                # models/, log/, metrics/, plots/
|   |-- evaluation/                  # The tables that compare runs
|   |-- plots/                       # The cross-run figures
|   `-- RESULTS.md
|-- app.py                           # Streamlit workstation
|-- AUDIT_REPORT.md                 # Code audit with dispositions
|-- PROJECT.md                      # Consolidated documentation
|-- README.md                       # Project overview
`-- report/
    `-- report.tex                  # This file
```

C Hyperparameter Sensitivity

The PPO hyperparameters in Table 3 are the Stable-Baselines3 defaults, with three exceptions: the network is widened to 2×256 units, the entropy coefficient is raised to 0.02 to keep the policy from collapsing to deterministic actions early, and the training runs on the CPU with one thread per process so that several policies can train in parallel on one machine. Measured

on this machine, PPO on this MLP runs at 812 steps/s on the CPU against 318 steps/s on the GPU, so the GPU is not used.

The entropy coefficient matters most: with too little of it the policy settles into a deterministic action early and stops exploring the envelope.

D Energy Model Derivation

The specific cutting energy $k_c(v_c)$ models the empirical observation that the energy required to remove a unit volume of material increases with cutting speed due to:

1. Higher shear zone temperatures $\rightarrow$ increased flow stress
2. Increased strain rates $\rightarrow$ strain-rate hardening
3. Tool-chip interface friction changes

The power-law form $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$ with $m \approx 0.6$ is a standard empirical form for the rise of the specific cutting energy with cutting speed, and it is the modelling choice of this work rather than a result taken from either reference paper: the reference papers optimize quality and efficiency, and treat energy only as related work. The reference condition v_{ref} is the cutting speed at the calibrated nominal parameters ($n = 5000$ RPM, $D = 50$ mm $\rightarrow v_{\text{ref}} \approx 785$ m/min). At this speed, $k_c = k_{c0} = 2.5$ W·min/mm^3.

Total power: $P_{\text{total}} = P_{\text{idle}} + k_c(v_c) \cdot \text{MRR}$. Energy per pass: $E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}}$. Substituting $t_{\text{cut}} = L_{\text{cut}}/(nf_r)$ and $\text{MRR} = v_c a_p f_r$:

$$E_{\text{pass}} = P_{\text{idle}} \frac{L_{\text{cut}}}{nf_r} + k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^m v_c a_p f_r \frac{L_{\text{cut}}}{nf_r} \tag{14}$$

$$= \underbrace{P_{\text{idle}} \frac{L_{\text{cut}}}{nf_r}}_{\text{Idle energy}} + \underbrace{k_{c0} \frac{L_{\text{cut}} a_p}{nf_r} \left(\frac{\pi D n}{1000 v_{\text{ref}}} \right)^m \frac{\pi D n}{1000}}_{\text{Cutting energy}} \tag{15}$$

The idle term $\propto 1/(nf_r)$ decreases with speed; the cutting term $\propto n^m$ increases with speed. Their sum has an interior minimum, which is the operating point selected by w_2.